



UTN.BA
UNIVERSIDAD TECNOLÓGICA NACIONAL
FACULTAD REGIONAL BUENOS AIRES

**Centro de
e-Learning**

UNIDAD DIDÁCTICA II

PYTHON PARA PRINCIPIANTES

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning

Modulo I – Introducción

Unidad I – VSC, debug y documentación.



Presentación:

En esta unidad nos ocuparemos de comenzar a trabajar con la plataforma de desarrollo de VSC de forma de poder beneficiarnos de temas como el autocompletado de código y el control de errores. También veremos cómo documentar nuestro código y seguiremos aprendiendo sobre operadores.



Objetivos:

Que los participantes:

Comiencen a utilizar la plataforma de desarrollo VSC.

Aprendan a realizar un seguimiento de errores en el código.

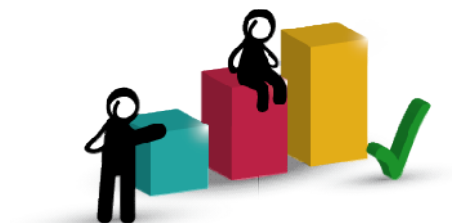
Comiencen a documentar sus trabajos.

Sigan aprendiendo sobre operadores.



Bloques temáticos:

- 1.- VSC.
- 2.- Autocompletado de código.
- 3.- ¿Cómo puedo seguir la ejecución de mi código?
- 4.- ¿Puedo ir documentando mi trabajo?
- 5.- PyDoc
- 6.- Operadores 2



Consignas para el aprendizaje colaborativo

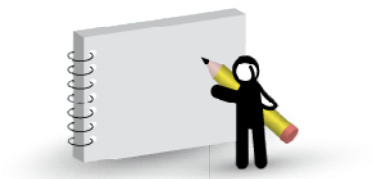
En esta Unidad los participantes se encontrarán con diferentes tipos de actividades que, en el marco de los fundamentos del MEC*, los referenciarán a tres comunidades de aprendizaje, que pondremos en funcionamiento en esta instancia de formación, a los efectos de aprovecharlas pedagógicamente:

- Los foros proactivos asociados a cada una de las unidades.
- La Web 2.0.
- Los contextos de desempeño de los participantes.

Es importante que todos los participantes realicen algunas de las actividades sugeridas y compartan en los foros los resultados obtenidos.

Además, también se propondrán reflexiones, notas especiales y vinculaciones a bibliografía y sitios web.

El carácter constructivista y colaborativo del MEC nos exige que todas las actividades realizadas por los participantes sean compartidas en los foros.



Tomen nota

Las actividades son opcionales y pueden realizarse en forma individual, pero siempre es deseable que se las realice en equipo, con la finalidad de estimular y favorecer el trabajo colaborativo y el aprendizaje entre pares. Tenga en cuenta que, si bien las actividades son opcionales, su realización es de vital importancia para el logro de los objetivos de aprendizaje de esta instancia de formación. Si su tiempo no le permite realizar todas las actividades, por lo menos realice alguna, es fundamental que lo haga. Si cada uno de los participantes realiza alguna, el foro, que es una instancia clave en este tipo de cursos, tendrá una actividad muy enriquecedora.

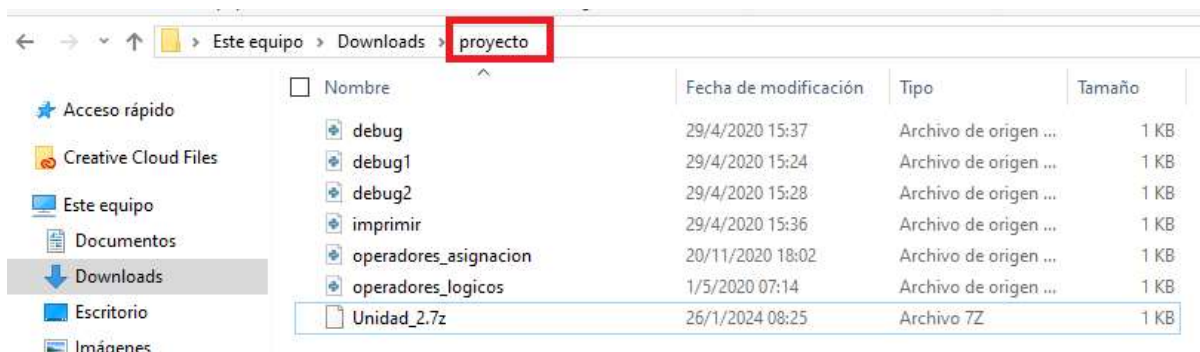
Asimismo, también tengan en cuenta cuando trabajen en la Web, que en ella hay de todo, cosas excelentes, muy buenas, buenas, regulares, malas y muy malas. Por eso, es necesario aplicar filtros críticos para que las investigaciones y búsquedas se encaminen a la excelencia. Si tienen dudas con alguno de los datos recolectados, no dejen de consultar al profesor-tutor. También aprovechen en el foro proactivo las opiniones de sus compañeros de curso y colegas.

1. VSC – Primer proyecto

Para comenzar, descargaremos de la unidad 2 los ejercicios con los que comenzaremos a trabajar:

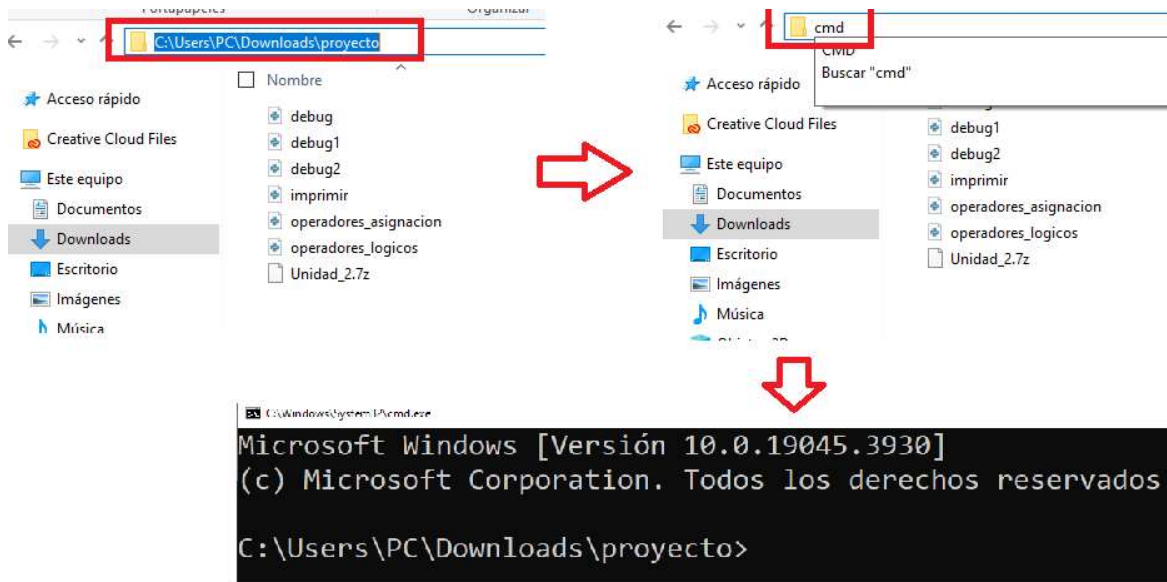


Luego creamos un directorio de trabajo dentro del cual descomprimos el .zip. En mi caso le he llamado “proyecto”.





Si nos paramos en la parte superior donde aparece la ruta del directorio, escribimos “cmd” y presionamos “ENTER”, se nos abre la terminal de Windows en el directorio donde tenemos nuestro proyecto.



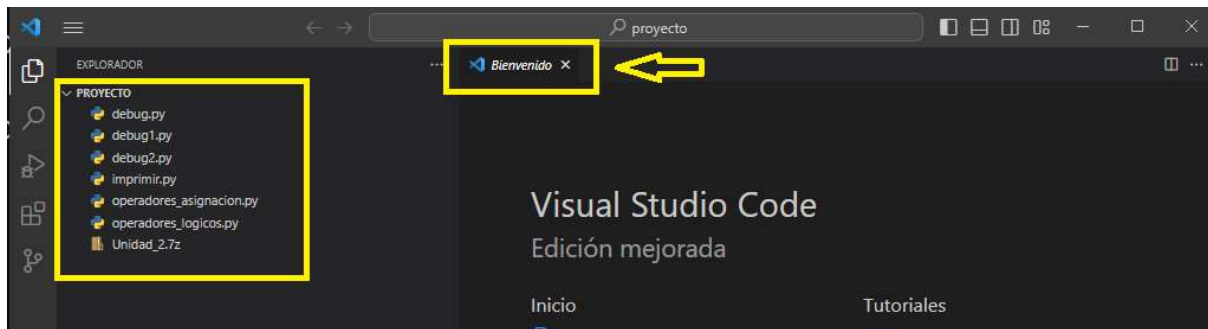
Ahora desde la terminal escribimos:

code .

Y presionamos “ENTER”, de forma que se nos abra el VSC desde el directorio de trabajo. Por ahora esto puede no parecer importante, pero si trabajamos con importación de archivos (módulos) es necesario ejecutar el editor desde el directorio en donde se encuentra nuestro archivo principal del programa para que no se presenten errores durante la ejecución. (Este punto sería conveniente consultárselo al docente durante la clase online para que pueda ampliar el tema)



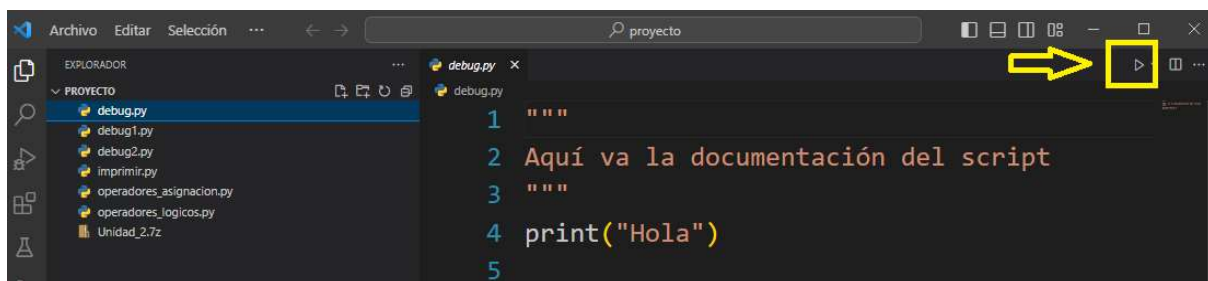
Al abrirlo de esta forma el VSC se abre presentando en el margen izquierdo nuestro proyecto, y en el margen derecho una página de bienvenida que podemos cerrar con la cruz indicada en la figura.



Si ahora hacemos doble click en cualquiera de los archivos del directorio este se abre:

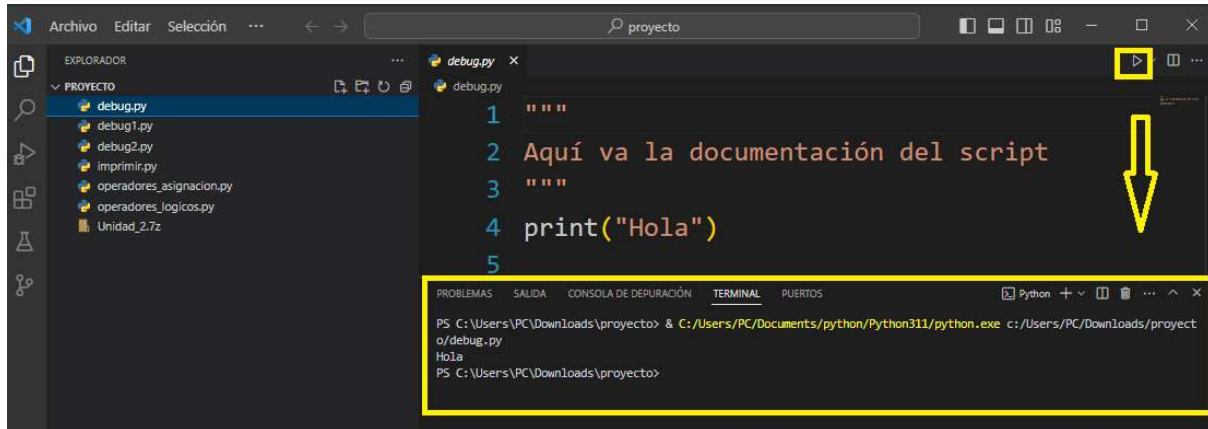


Desde el margen superior derecho podemos ejecutar el programa si presionamos el botón con forma de triángulo:



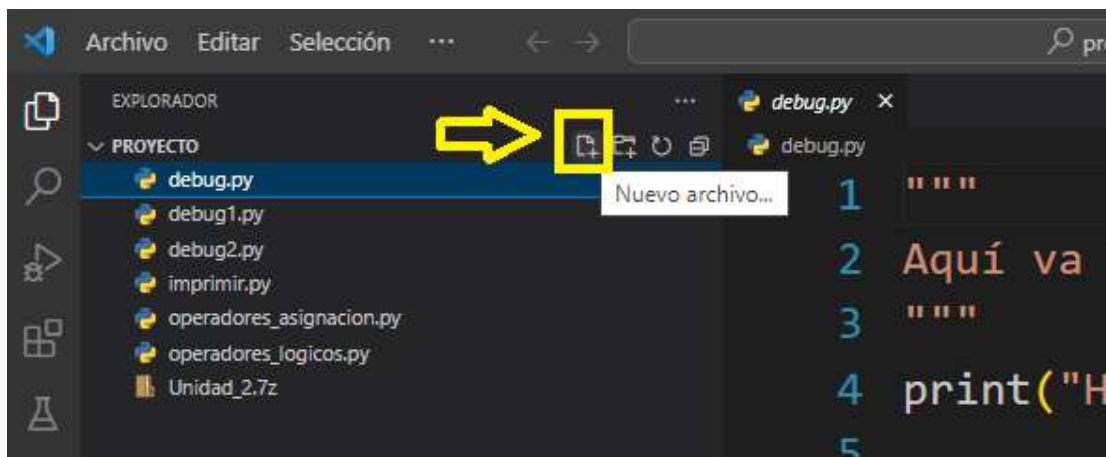


El resultado aparece en una terminal nueva que se abre durante la ejecución:



¿Cómo crear un script?

Para crear un nuevo script solo tenemos que presionar el ícono que aparece en la imagen. Al darle el nombre con extensión .py y presionar ENTER, VSC detecta el formato del contenido automáticamente.



El archivo se abre en la sección de edición por defecto, si no se abiera basta con presionar click izquierdo dos veces sobre el archivo en el panel de contenido para forzar la apertura. Pongámosle un código simple como:

debug1.py
print("Hola")

Ahora desde el menú superior ejecutarlo como hemos visto presionando el ícono con forma de triángulo.

2. Autocompletado de código

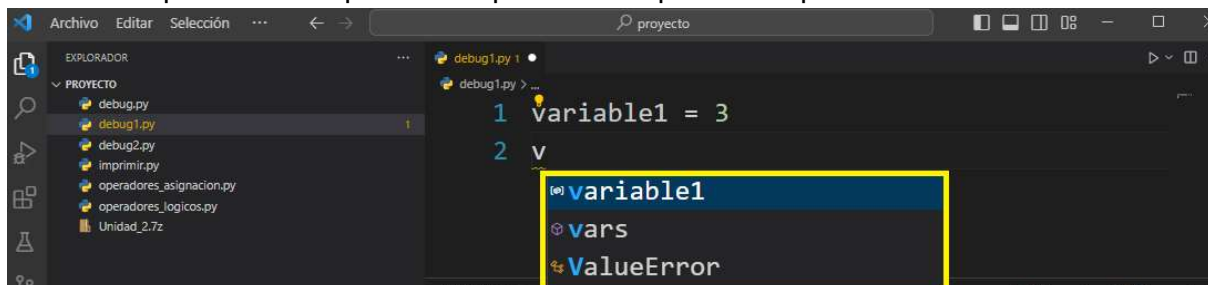
Una gran funcionalidad que nos brinda “VSC” es que nos ayuda con el autocompletado del código. Veamos cómo funciona paso a paso.

PASO 1

Crearemos un código que posea una variable apuntando a un valor entero.

debug1
1 variable1 = 3

Si ahora en la siguiente línea escribimos la letra “v”, podemos ver como VSC ya reconoce que la variable1 fue declarada y nos ofrece autocompletar el código desplegando una serie de opciones de palabras que son importante que inician con la letra “v”.



Para recorrer las opciones podemos utilizar las flechas del teclado, y para seleccionar la opción presionar “ENTER”.



3. ¿Cómo puedo seguir la ejecución de mi código?

Cuando trabajamos con un lenguaje de programación, es necesario poder identificar los posibles errores que cometemos al escribir código, de forma de poder solucionar los inconvenientes (Los cuales se suelen llamar **BUGS**). El proceso de identificación y solución de errores se suele conocer con el término de **Debugging**.

Cuando realizamos un **debugging** de un determinado código, tenemos la opción de realizarlo línea por línea de forma de poder analizar que puede llegar a estar funcionando mal en cada paso del programa.

Para ello VSC nos permite realizar un seguimiento, indicando la línea hasta la cual queremos que se ejecute el código, Veamos un ejemplo.

	debug2
1	variable1 = 2
2	variable2= 3
3	print("Hola 1")
4	print("Hola 2")
5	variable3 = variable1 * variable2
6	print(variable3)

Al ejecutarlo obtenemos como resultado:

Hola 1
Hola 2
6



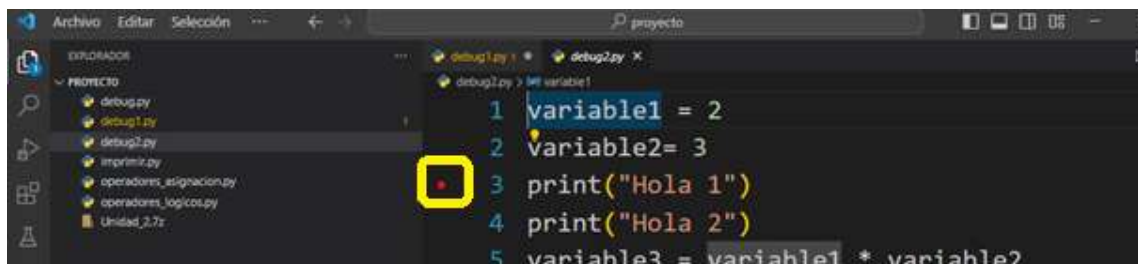
The screenshot shows the Visual Studio Code interface with a project named 'proyecto'. The Explorer panel on the left shows a file structure with 'PROYECTO' containing 'debug.py', 'debug1.py', 'debug2.py', 'imprimir.py', 'operadores_asignacion.py', 'operadores_logicos.py', and 'Unidad_2.7z'. The main editor window shows 'debug2.py' with the following code:

```
1 variable1 = 2
2 variable2= 3
3 print("Hola 1")
4 print("Hola 2")
5 variable3 = variable1 * variable2
6 print(variable3)
7 print("---"*21)
```

The output window at the bottom shows the execution results:

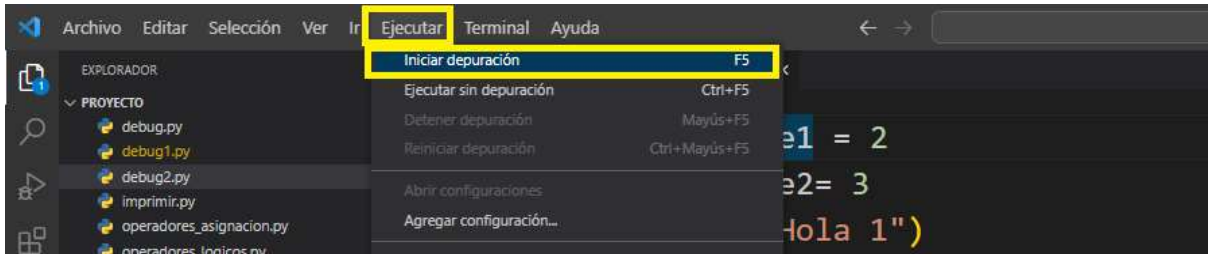
```
PS C:\Users\PC\Downloads\proyecto> & C:\Users\PC\Documents\python\Python311\python.exe c:/Users/PC/Downloads/proyecto/debug2.py
Hola 1
Hola 2
6
-----
PS C:\Users\PC\Downloads\proyecto>
```

Ahora bien, pongamos un punto de control en la segunda línea, para ello nos pararemos al inicio de la línea 2 y presionaremos click izquierdo sobre el margen izquierdo, como se muestra a continuación:

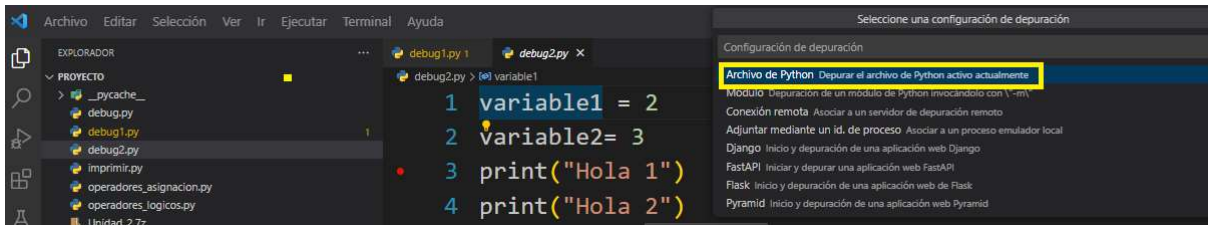




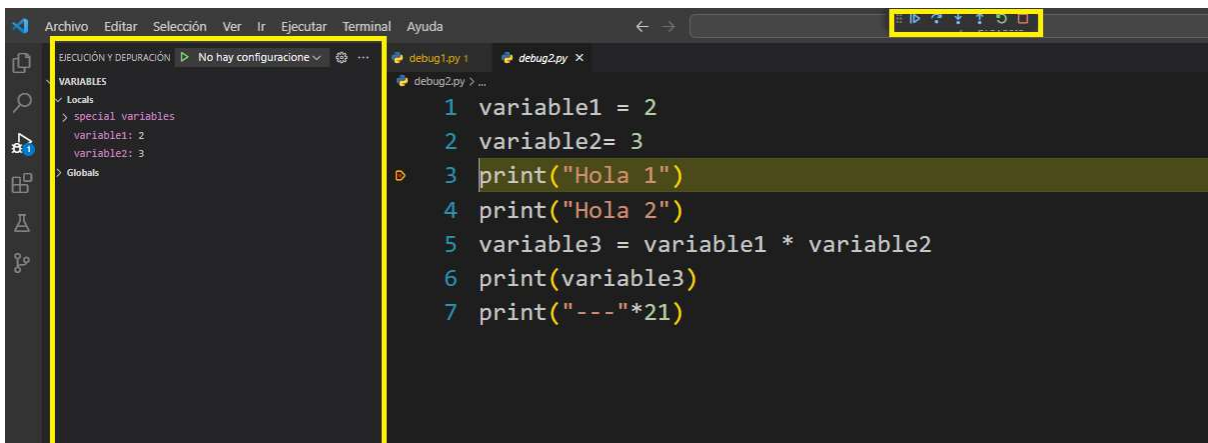
Luego desde el menú superior seleccionamos: **Ejecutar > Iniciar depuración**



Se despliegan algunas opciones, en donde seleccionamos: **Archivo de Python**



Ahora nuestra ventana se modifica y ya nos muestra información sobre lo que se ha ejecutado del código hasta la línea 3:



Se puede notar que a esta altura ya reconoce que la variable 1 vale 2 y que la variable 2 vale 3. Podemos agregar más de un punto de control y avanzar y retroceder con los íconos que aparecen en el menú superior.



En el siguiente caso se ven agregados tres puntos de control (líneas 2, 4 y 7) que pueden ser avanzados.

Las salidas nos retornan:

Hasta punto 1		
<pre>▼ VARIABLES ▼ Locals > special variables variable1: 2</pre>		Hasta el primer punto la variable 2 aún no ha sido definida.
Hasta punto 2		
<pre>▼ VARIABLES ▼ Locals > special variables variable1: 2 variable2: 3</pre>		En el segundo punto ya se ha definido la variable 2 y se le ha asignado valor.
Hasta punto 3		
<pre>▼ VARIABLES ▼ Locals > special variables variable1: 2 variable2: 3 variable3: 6</pre>		En el tercer punto ya se ha definido la variable 3 que es el resultado de la multiplicación de las dos anteriores.

Nota: El desplegable “**Special Variables**”, tiene un montón de información que no estamos en condiciones de entender a esta altura, pero que es muy importante en el análisis del código.



4. ¿Puedo ir documentando mi trabajo?

Podemos encontrar documentación sobre un programa que estemos analizando en diferentes fuentes tanto dentro como fuera del documento. En la unidad 1 vimos como podíamos documentar nuestro código con comentarios, tanto de una como de multi líneas. Los comentarios, si se encuentran al inicio del programa pueden ser visualizados desde otro script con la ayuda de la rutina **help()**.

Veamos un ejemplo en donde considerados dos archivos, que se encuentran en un mismo directorio de nuestro sistema operativo. Supongamos que agregamos una descripción al archivo "**debug.py**" y lo importamos desde el archivo "**importar.py**" como se muestra a continuación:

	debug.py
1	"""
2	Aquí va la documentación del script
	"""
	print("Hola")

	importar.py
1	import debug
2	print(help(debug))

La salida de ejecutar importar.py nos retorna:

```
Hola
Help on module debug:

NAME
  debug - Aquí va la documentación del script

FILE
  c:\users\juanb\documents\medrano\001-python_paso_a_paso\unidad_2\debug.py

None

Process finished with exit code 0
```

Como podemos ver debajo del el atributo “NAME” nos figura el comentario agregado.

5.- PyDoc

Al instalar la distribución de Python, se nos instala pydoc, esta utilidad, nos permite recuperar y buscar rápidamente documentación desde el Shell así como crear páginas html de documentación.

Ver opciones de comandos

Si abro un Shell y ejecuto:

`python -m pydoc`

Nos aparecen todas las opciones de comandos que podemos utilizar para obtener información de las estructuras que podemos crear dentro de nuestro código, y crear nuestra propia documentación.

Veamos cada caso que nos aparece como opción al ejecutar el comando anterior:

`pydoc <name> ...`

Permite mostrar documentación de texto sobre algo. <nombre> puede ser el nombre de una palabra clave, tema, función, módulo o paquete de Python, o una referencia punteada a una clase o función dentro de un módulo o módulo en el paquete. Si <nombre> contiene un '\', se usa como la ruta a un archivo fuente de Python para documentar. Si el nombre es 'palabras clave', 'temas' o 'módulos', se muestra una lista de estas cosas.

Como ejemplo podemos consultar información sobre el módulo “random” el cual nos permite generar números aleatorios, de la siguiente forma:

`python -m pydoc random`



Help on module random:

NAME

random - Random variable generators.

DESCRIPTION

integers

uniform within range

sequences

pick random element

pick random sample

pick weighted random sample

generate random permutation

distributions on the real line:

uniform

triangular

normal (Gaussian)

lognormal

negative exponential

gamma

beta

pareto

Weibull

-- Más --

Como podemos ver al final de la información que se despliega en el Shell nos aparece la opción de ver más, con lo cual si presionamos “**Enter**” nos seguirá apareciendo más información la cual se irá desplegando de a una línea por vez, es decir que cada “**Enter**” equivale a una nueva línea de descripción.



pydoc -k <keyword>

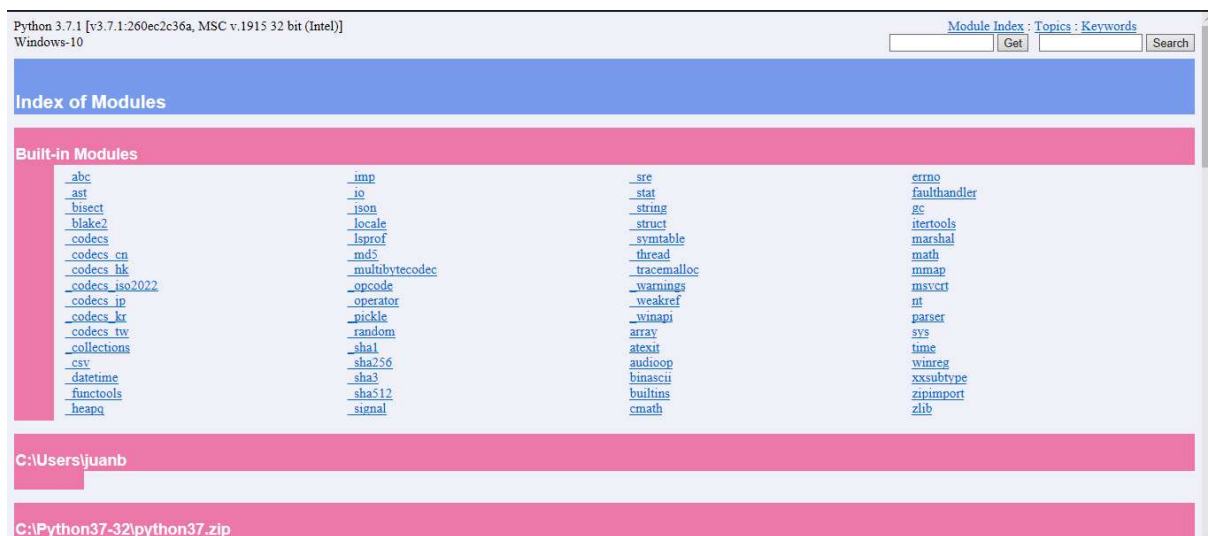
Permite buscar una palabra clave en las líneas de sinopsis de todos los módulos disponibles.

pydoc -n <hostname>

Inicia un servidor HTTP con el número de puerto dado, (en este caso 54200)

```
C:\Users\juanb>python -m pydoc -p 54200
Server ready at http://localhost:54200/
Server commands: [b]rowser, [q]uit
server>
```

Por lo que si ingresamos en la dirección dada en un explorador podemos acceder a la página desde la cual encontramos el listado de todos los módulos de nuestra distribución.



pydoc -b

Inicia un servidor HTTP en un puerto arbitrario no utilizado y abre un navegador web para navegar interactivamente por la documentación. Esta opción se puede utilizar en combinación con -n y / o -p.



pydoc -w <name> ...

Permite crear una página html en base a la información agregada a los objetos de un determinado archivo. Podemos probarlo con nuestro ejercicio debug2.py, si accedemos al directorio en donde este se encuentra desde el cmd y ejecutamos:

python -m pydoc -w debug2

Esto nos generará una página web con el mismo nombre conteniendo la información de nuestra programa, la cual se agregará en el mismo directorio en el cual tenemos el archivo.

Podemos hacer doble click sobre el archivo debug2.html que se acaba de crear para que la página web se abra.



6. Operadores 2

Grupo 3 – Operadores de Asignación

Los operadores de asignación permiten asignar valores a variables, en la siguiente tabla se recogen algunos de los operadores más utilizados, en su forma abreviada y equivalente.

Operador	Ejemplo	Es equivalente a:
=	x = 7	x = 7
+=	x += 7	x = x + 7
-=	x -= 7	x = x - 7
*=	x *= 7	x = x * 7
/=	x /= 7	x = x / 7
%=	x %= 7	x = x % 7
//=	x //= 7	x = x // 7
**=	x **= 7	x = x ** 7

Grupo 4 – Operadores Lógicos

Los operadores lógicos son:

Operador	Descripción
and	Es verdadero si ambas condiciones se cumplen
or	Es verdadero si una de las condiciones es se cumplen
not	Es verdadero si ninguna condición se cumple,,

Ejemplo:

	operadores_logicos.py
1	a, b, c, d, e, f = 2, 4, 5, 21, 22, 3
2	print(a == 2 and b == 2)
3	print(a == 2 or b == 2)

Existen también los operadores:



- Bit a bit
- De identificación
- De membrecía

Estos operadores no tienen mucho sentido de introducirlos a esta altura, por lo que el alumno que desee puede preguntar por ellos al docente si quiere o necesita ampliar su información.

Estructuras de control y bucles

Otro componente fundamental en nuestros scripts lo constituyen los bucles y las estructuras de control, diseñados para cumplir con la tarea de tomar decisiones, seleccionar elementos, etc. Todos los lenguajes de programación cuentan con estas herramientas aún cuando su sintaxis puede variar un poco entre lenguajes. A continuación veremos el uso de: if – if/else y for

if if/elif/else

La estructura if nos sirve para tomar decisiones, su estructura básica es como sigue:

```
if x:
    if y:
        Declaración1
    else:
        Declaración2
```

El código anterior se lee como: si x es verdadero; si y es verdadero entonces se cumple el código “Declaración 1” , sino se cumple el código “Declaración 2”.

Notar que como regla:

- La condición no va entre paréntesis.
- Los finales de línea no llevan punto y coma.
- El código de cada sección no lleva delimitadores.

Centro de e-Learning SCEU UTN - BA.

Medrano 951 2do piso (1179) // Tel. +54 11 4867 7589 / Fax +54 11 4032 0148

www.sceu.frba.utn.edu.ar/e-learning



- Se indica a quién pertenece por medio de la indentación.
- No tiene por qué ser una tabulación, puede ser espacios pero siempre deben ser la misma cantidad para cada declaración
- Las declaraciones pueden ponerse en una misma línea si se separan por punto y coma

Uso de elif

El uso de elif nos permite evaluar varias condiciones, su estructura de uso es como sigue:

```
if condición1:
```

```
    Código1
```

```
elif condición2:
```

```
    Código2
```

```
else:
```

```
    Código3
```

for

La estructura for, es mucho más rica que en otros lenguajes de programación, por lo que más adelante daremos un análisis completo de esta estructura. Por ahora y para comenzar a realizar ejercicios simples vamos a definir su forma básica como sigue:

```
for objetivo in objeto:
```

```
    código1
```

```
else:
```

```
    código2
```



Esta forma poco típica de utilización de un bucle for puede sorprender a quien viene de otros lenguajes de programación, sin embargo es de uso frecuente en Python y nos permite ejecutar un código dependiendo de si un determinado objeto se encuentra dentro de otro o no.

While

El bucle while nos permite ejecutar ciclos, mientras se cumpla una determinada condición. La estructura es:

```
while (condición):  
    contenido
```

Un ejemplo puede ser:

```
valor = 3  
total = 0  
while valor <= 7:  
    total = valor + total  
    valor = valor + 1  
print (total)
```



Bibliografía utilizada y sugerida

Libros

Programming Python 5th Edition – Mark Lutz – O'Reilly 2013

Programming Python 4th Edition – Mark Lutz – O'Reilly 2011

Manual online

<https://docs.python.org/3.7/tutorial/>



Lo que vimos

En esta unidad hemos comenzado a trabajar con Pycharm, realizado un seguimiento de errores en nuestro código y visto como obtener una documentación de nuestro código.



Lo que viene:

En la siguiente unidad veremos como utilizar pandas para la adquisición de datos y Matplotlib para la representación visual.